

Настройка приложений и управление доступом

Кирилл Касаткин
DevOps-инженер, Renue

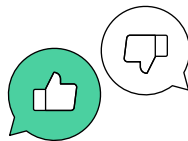


Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти



Поставьте в чат:

-  если меня видно и слышно
-  если нет

Рекомендации

- Если смотрите с компьютера
 - Используйте браузеры **Google Chrome** или **Microsoft Edge**
 - Если есть проблемы с изображением или звуком, обновите страницу — **F5**

- Если смотрите с мобильного телефона или планшета
 - Перейдите с мобильного интернет-соединения на **Wi-Fi**
 - Если есть проблемы с изображением или звуком, перезапустите приложение **МТМ-Линк** на телефоне

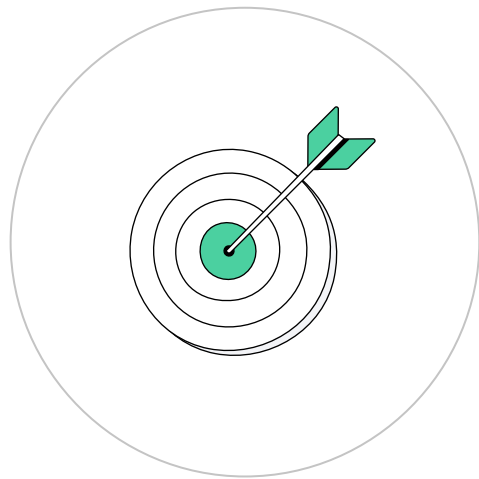
Кирилл Касаткин

DevOps-инженер, Renue



Цели занятия

- Узнать:
 - что такое ConfigMap, Secret и чем они отличаются
 - как разделить и хранить конфигурации от Pod'ов
- Познакомиться со способами применения конфигураций
- Узнать:
 - что такое RBAC
 - что такое ServiceAccount и Роли (Role)
- Разобраться с подключением Ролей и пользователей
- Разобрать примеры манифестов объектов K8s
- Узнать механизмы выдачи доступа к кластеру k8s

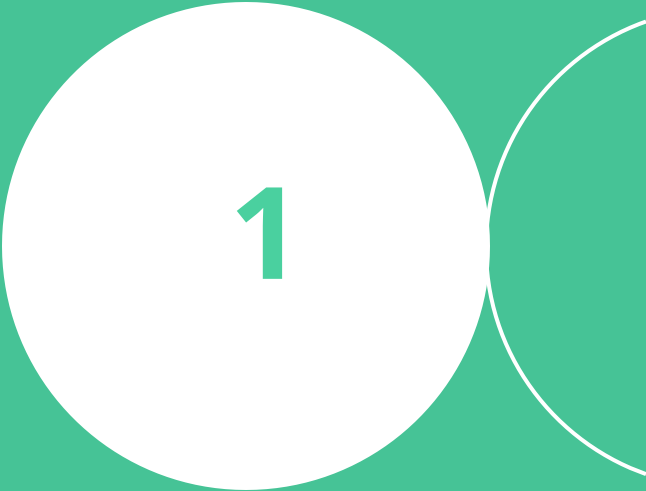


План занятия

- 1 Конфигурация приложения
- 2 ConfigMaps
- 3 Secrets
- 4 Доступ к API K8s
- 5 RBAC
- 6 Итоги занятия



Конфигурация приложения



1



**Конфигурация приложения
— это возможность
динамического добавления
параметров приложению**

Конфигурация приложения

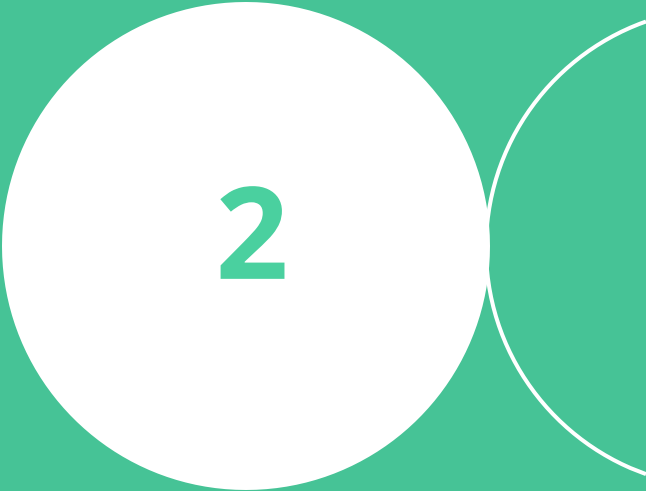
Принято разделять конфигурационные файлы и контейнеры с приложениями для того, чтобы избавиться от необходимости упаковывать конфигурации в image приложения и обеспечить отказоустойчивость и безопасность

В зависимости от содержимого есть 2 типа конфигураций :

→ ConfigMap

→ Secret

ConfigMaps



2



**ConfigMaps — объект k8s,
который позволяет
хранить нечувствительные
конфигурации в формате
ключ:значение**

Пример конфигурации

Пример конфигураций ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-configmap
  namespace: my-ns
data:
  key1: value1
  key2: value2
  key3:
    subkey: somevalue
  key4: |
    Test
    multiple lines
    more lines
```

- metadata — имя и namespace



Пример конфигурации

Пример конфигураций ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-configmap
  namespace: my-ns
data:
  key1: value1
  key2: value2
  key3:
    subkey: somevalue
  key4: |
    Test
    multiple lines
    more lines
```

- metadata — имя и namespace
- Данные могут быть в виде
 - ключ: значение



Пример конфигурации

Пример конфигураций ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-configmap
  namespace: my-ns
data:
  key1:      value1
  key2:      value2
  key3:
    subkey: somevalue
  key4: |
    Test
    multiple lines
    more lines
```

- metadata — имя и namespace
- Данные могут быть в виде
 - ключ: значение
 - ключ: вложенные (ключ:значение)



Пример конфигурации

Пример конфигураций ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-configmap
  namespace: my-ns
data:
  key1:      value1
  key2:      value2
  key3:
    subkey: somevalue
  key4: |
    Test
    multiple lines
    more lines
```

- metadata — имя и namespace
- Данные могут быть в виде
 - ключ: значение
 - ключ: вложенные (ключ:значение)
 - ключ: текст



Пример конфигурации

Подключение ConfigMap к Pod происходит несколькими способами: **ENV**

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-configmap
  namespace: my-ns
data:
  key1: value1
  key2: value2
  key3:
    subkey: somevalue
  key4: |
    Test
    multiple lines
    more lines
```

```
apiVersion: v1
kind: Pod metadata:
  name: env-pod
  namespace: my-ns
spec:
  containers:
    - name: busybox
      image: busybox env:
        - name: CONFIGMAPVAR
          valueFrom:
            configMapKeyRef:
              name: my-configmap
              key: key1
```



Пример конфигурации

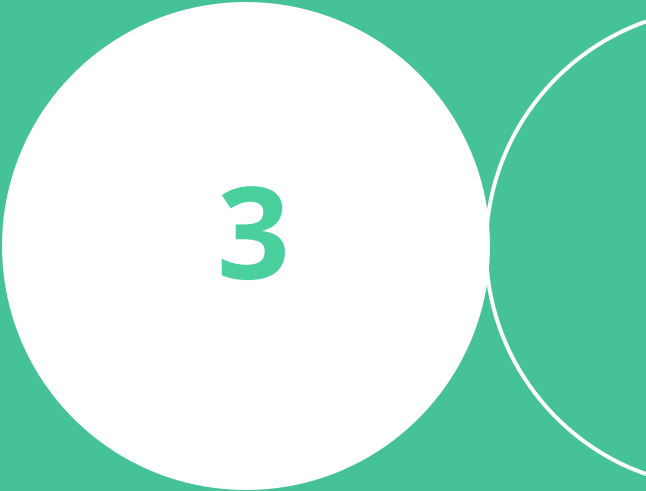
Подключение ConfigMap к Pod происходит несколькими способами: **Volume**

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-configmap
  namespace: my-ns
data:
  key1: value1
  key2: value2
  key3:
    subkey: somevalue
  key4: |
    Test
    multiple lines
    more lines
```

```
apiVersion: v1
kind: Pod
metadata:
  name: vol-pod
  namespace: my-ns
spec:
  containers:
    - name: busybox
      image: busybox
      volumeMounts:
        - name: configmap-volume
          mountPath: /etc/config/configmap
  volumes:
    - name: configmap-volume
      configMap:
        name: my-configmap
```



Secrets



3



**Secret — объект k8s,
который позволяет хранить
конфиденциальные
конфигурации в формате
ключ:значение**

Пример конфигурации

Пример конфигураций Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
  namespace: my-ns
type: Opaque
data:
  username: <base64 String 1>
  password: <base64 String 2>
```

- metadata — имя и namespace
- значение должно храниться в кодировке base64



Пример конфигурации

Подключение Secret к Pod происходит несколькими способами: **ENV**

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
  namespace: my-ns
type: Opaque
data:
  username: <base64 String 1>
  password: <base64 String 2>
```

```
apiVersion: v1
kind: Pod
metadata:
  name: env-pod
  namespace: my-ns
spec:
  containers:
    - name: busybox
      image: busybox
      env:
        - name: SECRETVAR
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: username
```



Пример конфигурации

Подключение Secret к Pod происходит несколькими способами: **Volume**

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
  namespace: my-ns
type: opaque
data:
  username: <base64 String 1>
  password: <base64 String 2>
```

```
apiVersion: v1
kind: Pod
metadata:
  name: vol-pod
  namespace: my-ns
spec:
  containers:
    - name: busybox
      image: busybox
      volumeMounts:
        - name: secret-volume
          mountPath: /etc/config/secret
  volumes:
    - name: secret-volume
      secret:
        secretName: my-secret
```



Пример конфигурации

Типы конфигураций Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
  namespace: my-ns
type: Opaque
data:
  username: <base64 String 1>
  password: <base64 String 2>
```

- **opaque (generic)** – определяемый пользователем
- **tls** – секрет из пары открытого и закрытого ключей
- **docker-registry** – секрет для доступа к хранилищу образов Docker
- и другие



Пример конфигурации

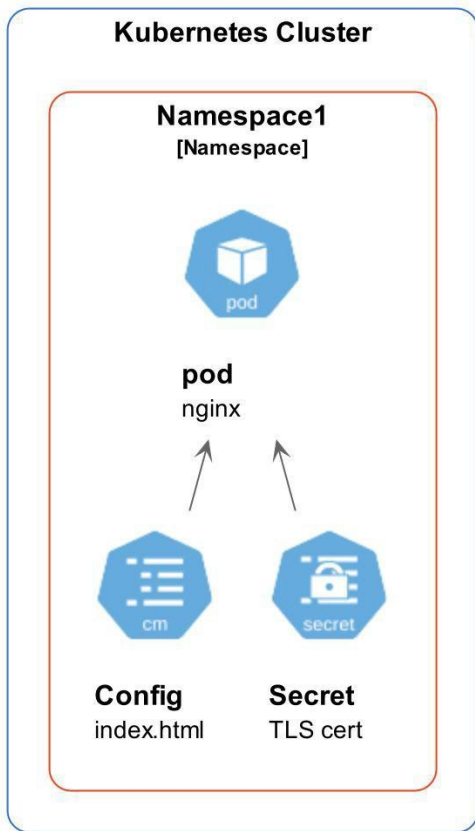
Примеры конфигураций Ingress TLS

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  rules:
    - host: my-app.com
      http:
        paths:
          - path: /
            backend:
              service:
                name: my-app-service
                port:
                  number: 80
  tls:
    - hosts:
        - my-app.com
      secretName: my-app-secret-tls
```

```
apiVersion: v1
kind: Secret
metadata:
  name: my-app-secret-tls
data:
  tls.crt: base64 encode cert
  tls.key: base64 encode key
type: kubernetes.io/tls
```



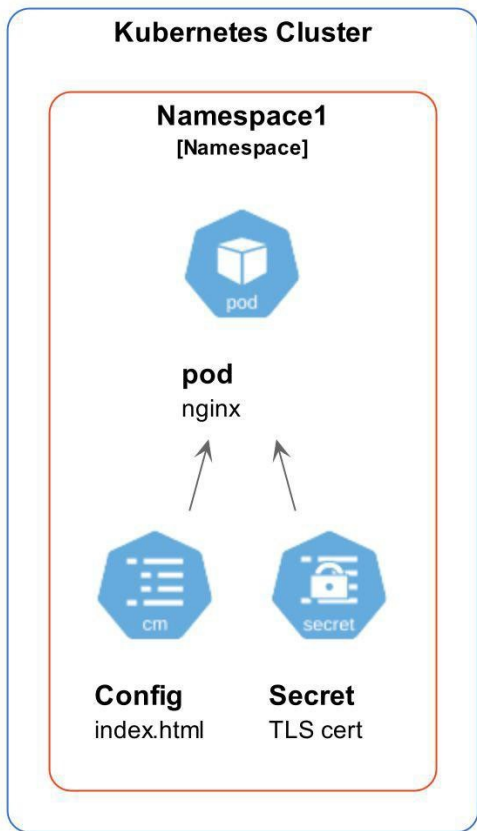
Использование Secret и ConfigMap



```
apiVersion: v1
kind: Pod
metadata:
  name: vol1-pod
  namespace: my-ns
spec:
  containers:
    - name: busybox
      image: busybox
      env:
        - name: CONFIGMAPVAR
          valueFrom:
            configMapKeyRef:
              name: my-configmap
              key: key1
        - name: SECRETVAR
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: username
```



Использование Secret и ConfigMap

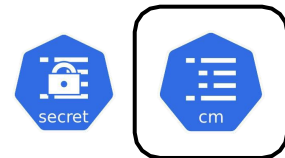


```
apiVersion: v1
kind: Pod
metadata:
  name: vol-pod
  namespace: my-ns
spec:
  containers:
  - name: busybox
    image: busybox
    volumeMounts:
    - name: configmap-volume mountPath:
      /etc/config/configmap
    - name: secret-volume mountPath:
      /etc/config/secret
  volumes:
  - name: configmap-volume
    configMap:
      name: my-configmap
  - name: secret-volume
    secret:
      secretName: my-secret
```



Особенности ConfigMap и Secret

- Должны быть созданы до того, как они будут использованы в модулях.
Ссылки на несуществующие объекты предотвратят запуск Pod



Особенности ConfigMap и Secret

- Должны быть созданы до того, как они будут использованы в модулях. Ссылки на несуществующие объекты предотвратят запуск Pod
- Смонтированные конфигурации обновляются *автоматически*, но при этом не все приложения умеют перечитывать настройки, а также в случае переменных среды требуется перезапуск модуля



Особенности ConfigMap и Secret

- Должны быть созданы до того, как они будут использованы в модулях. Ссылки на несуществующие объекты предотвратят запуск Pod
- Смонтированные конфигурации обновляются *автоматически*, но при этом не все приложения умеют перечитывать настройки, а также в случае переменных среды требуется перезапуск модуля
- Динамические настройки зависят от множества переменных. Возможно применение шаблонизатора (например, Jinja)



Особенности ConfigMap и Secret

- Должны быть созданы до того, как они будут использованы в модулях. Ссылки на несуществующие объекты предотвратят запуск Pod
- Смонтированные конфигурации обновляются *автоматически*, но при этом не все приложения умеют перечитывать настройки, а также в случае переменных среды требуется перезапуск модуля
- Динамические настройки зависят от множества переменных. Возможно применение шаблонизатора (например, Jinja)
- Размер не более 1MB. Если в base64, то не более 750kB



Особенности ConfigMap и Secret

- Должны быть созданы до того, как они будут использованы в модулях. Ссылки на несуществующие объекты предотвратят запуск Pod
- Смонтированные конфигурации обновляются *автоматически*, но при этом не все приложения умеют перечитывать настройки, а также в случае переменных среды требуется перезапуск модуля
- Динамические настройки зависят от множества переменных. Возможно применение шаблонизатора (например, Jinja)
- Размер не более 1MB. Если в base64, то не более 750kB
- Создание множества мелких конфигураций может истощить память



Особенности ConfigMap и Secret

- Должны быть созданы до того, как они будут использованы в модулях. Ссылки на несуществующие объекты предотвратят запуск Pod
- Смонтированные конфигурации обновляются *автоматически*, но при этом не все приложения умеют перечитывать настройки, а также в случае переменных среды требуется перезапуск модуля
- Динамические настройки зависят от множества переменных. Возможно применение шаблонизатора (например, Jinja)
- Размер не более 1MB. Если в base64, то не более 750kB
- Создание множества мелких конфигураций может истощить память
- Находятся в пространстве имён, что означает доступ только из того же пространства имён

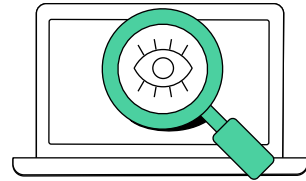




Ваши вопросы?

Демонстрация работы

Работа с ConfigMap и Secret



Закрепляем

Вопрос: Какой вариант конфигурации лучше всего подойдет для подключения к поду конфигурации nginx?



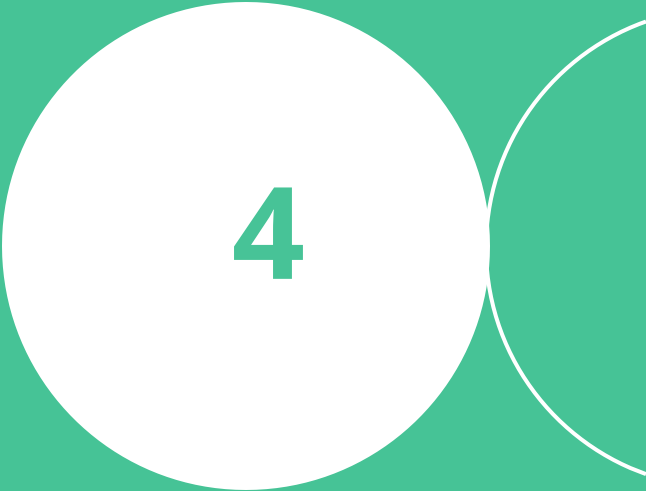
Закрепляем

Вопрос: Какой вариант конфигурации лучше всего подойдет для подключения к поду конфигурации nginx?

Ответ: Если в конфигурации отсутствуют чувствительные данные, то это будет configmap в виде volume



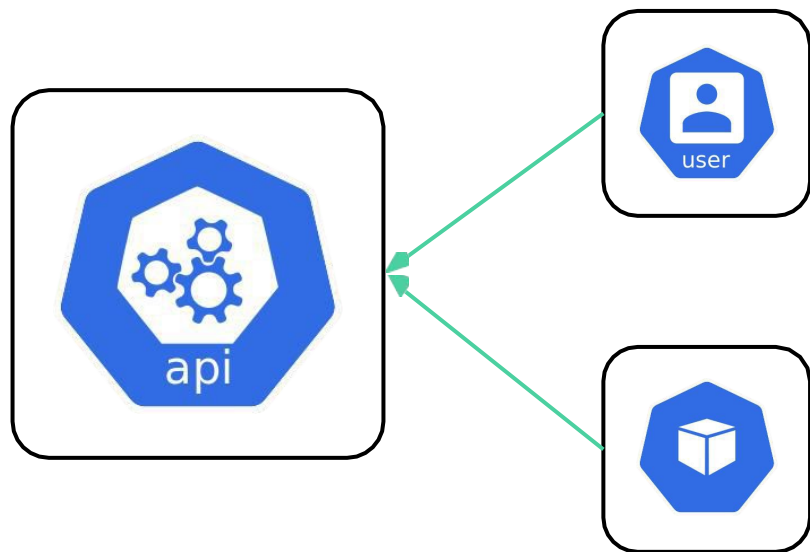
Доступ к API K8s



4

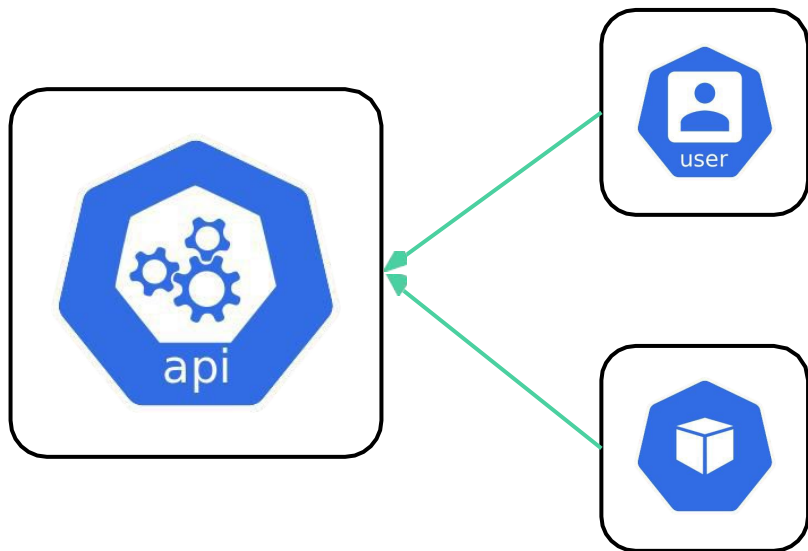
Авторизация и аутентификация

Доступ к API кластера K8s возможен как для пользователей, так и для приложений.
При этом K8s не управляет user accounts нативно (нет такого объекта)



Авторизация и аутентификация

Доступ к API кластера K8s возможен как для пользователей, так и для приложений. При этом K8s не управляет user accounts нативно (нет такого объекта)



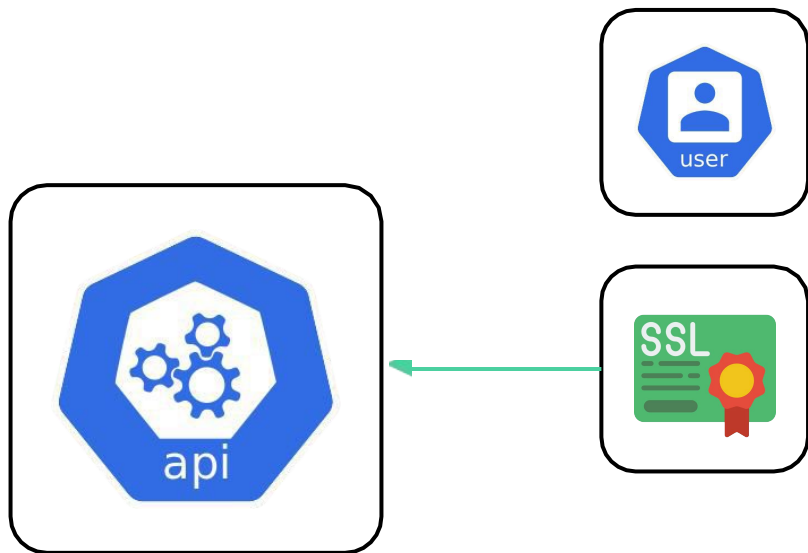
Авторизация пользователей возможна с помощью:

- базовой авторизации
- SSL-сертификатов
- token
- сторонних приложений

Авторизация приложений — token

Авторизация и аутентификация

Доступ к API кластера K8s для пользователя с помощью **SSL**



Особенности:

- можно сгенерировать сертификат и подписать в K8s
- можно использовать объект **CertificateSigningRequest** и `kubectl certificate approve` (допустим, со стороны админов)

После этого создать config-файл для подключения к кластеру (`kubectl config`)

Пример конфигурации

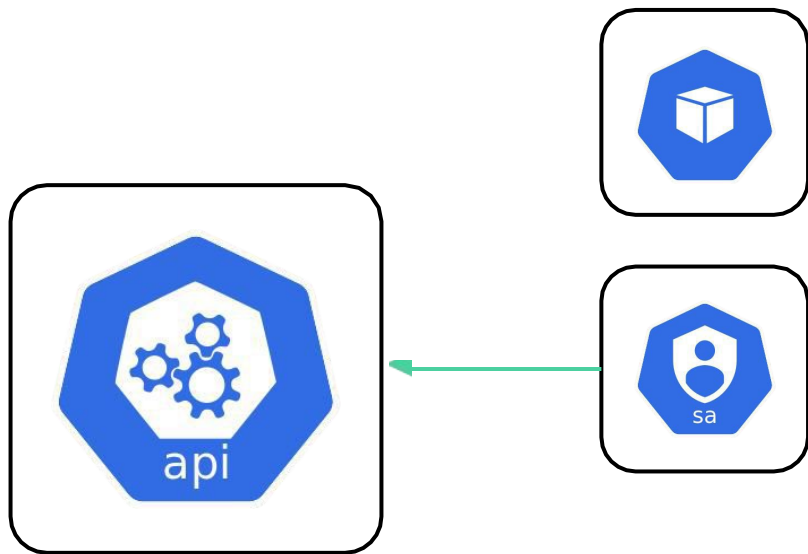
Пример конфигураций CertificateSigningRequest (подробности [здесь](#)):

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: ssl-csr
spec:
  groups:
  - system:authenticated
  request: ${BASE64_CSR}
  usages:
  - digital signature
  - key encipherment
  - server auth
  - client auth
```

- `kubectl get csr`
- `kubectl certificate approve ssl-csr`
- `kubectl get csr ssl-csr -o jsonpath={.status.certificate} | base64 --decode > cert.crt`

Авторизация и аутентификация

Доступ к API кластера K8s для Pod осуществляется с помощью **ServiceAccount** (сервис-аккаунт)



Особенности сервис-аккаунта:

- 1 Создается в namespace (по умолчанию default)
- 2 У каждого есть токен
- 3 Монтируется внутри контейнера в

`/var/run/secrets/kubernetes.io/serviceaccount:`

- ca.crt — сертификат CA кластера
- namespace содержит имя namespace, в котором находится Pod
- token содержит токен, используемый для доступа к API кластера

Пример конфигурации

Пример конфигураций ServiceAccount и Pod:

```
apiVersion:v1
kind: ServiceAccount
metadata:
  name: sa-pod
  namespace: my-ns
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod
  namespace: my-ns
metadata:
  containers:
    - name: busybox
      image: busybox
      command: ['some', 'command']
  serviceAccountName: sa-pod
```

RBAC



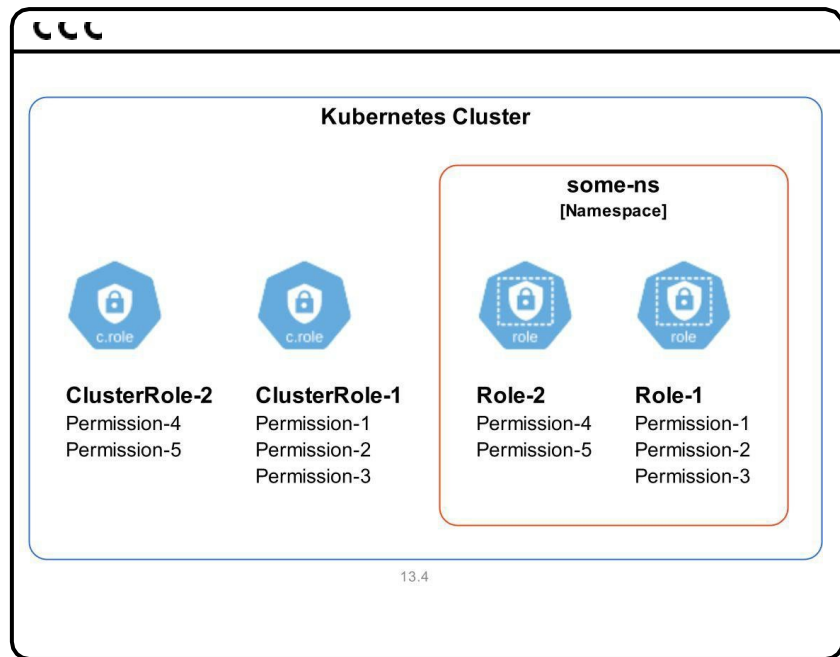
5

The background features three large, light gray circles that overlap each other. The central circle is the most prominent, and the text is centered within it. The other two circles are partially visible on the left and right sides of the frame.

**Role-based access
control (RBAC) —
управление доступом,
основанное на ролях**

Роли в RBAC

RBAC можно использовать со всеми ресурсами Kubernetes, которые поддерживают CRUD (create, read, update, delete)



- **Role** — используется для предоставления доступа к ресурсам внутри namespace
- **ClusterRole** используется для предоставления доступа к ресурсам, доступным в пределах всего кластера

Пример конфигурации

Пример конфигураций Role и ClusterRole:

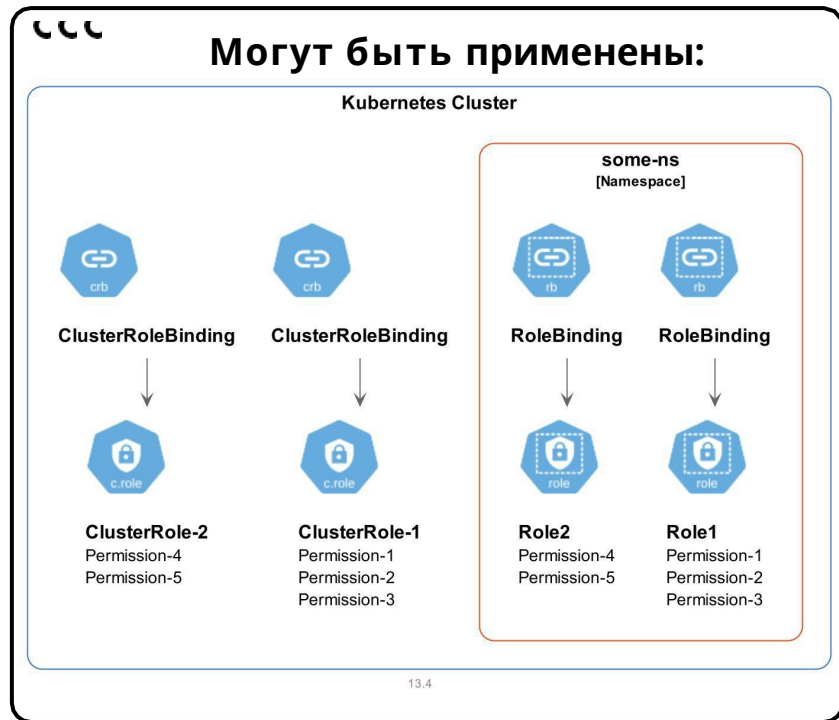
```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: my-ns
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "watch", "list"]
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["secrets"]
  verbs: ["get", "watch", "list"]
```

Список apiGroups можно увидеть командой *kubectl api-resources*

Роли в RBAC

RoleBinding и ClusterRoleBinding — привязка субъекта к Role или ClusterRole



- на users
- groups
- service accounts

Пример конфигурации

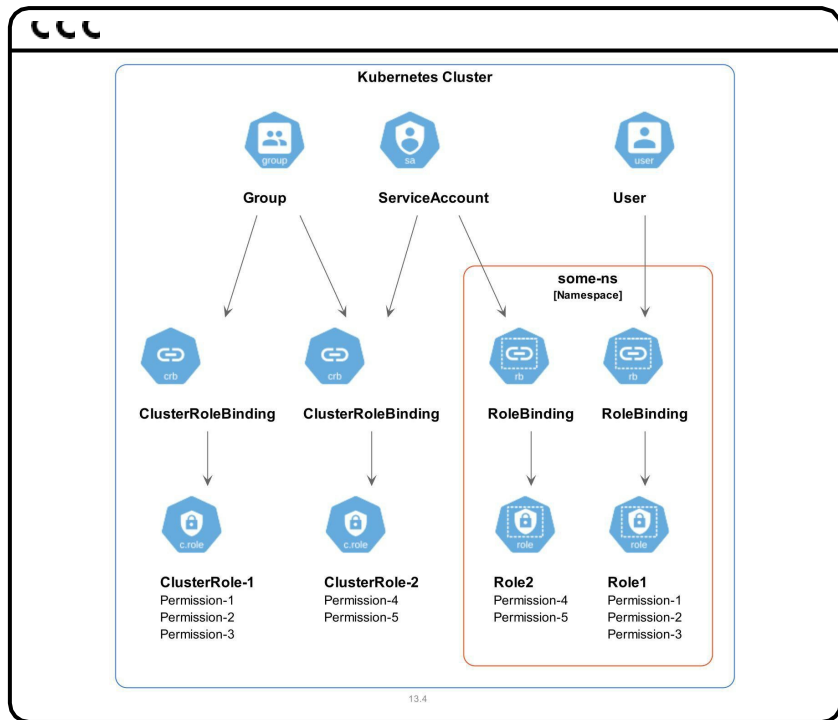
Пример конфигураций Role и ClusterRole:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader
  namespace: my-ns
subjects:
- kind: User
  name: dev
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

```
apiVersion: rbac.authorization.k8s.io/v1 kind: ClusterRoleBinding
metadata:
  name: read-secrets-global
subjects:
- kind: Group
  name: managers
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: secret-reader
  apiGroup: rbac.authorization.k8s.io
```

Роли в RBAC

RoleBinding и ClusterRoleBinding могут содержать несколько «пользователей»:



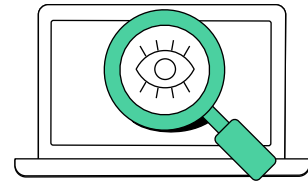
```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: read-secrets-global
subjects:
- kind: Group
  name: managers
  apiGroup: rbac.authorization.k8s.io
- kind: ServiceAccount
  name: sa-secret-reader
roleRef:
  kind: ClusterRole
  name: secret-reader
  apiGroup: rbac.authorization.k8s.io
```



Ваши вопросы?

Демонстрация работы

SSL-авторизация, ServiceAccount, RBAC



Закрепляем

Вопрос: Для каких целей мы используем механизм RBAC в рамках работы с k8s?



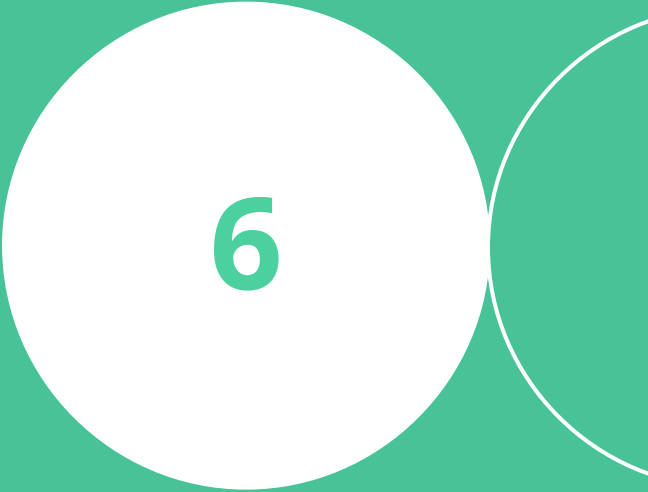
Закрепляем

Вопрос: Для каких целей мы используем механизм RBAC в рамках работы с k8s?

Ответ: Основной целью является разграничение доступа и улучшение безопасности



Итоги занятия



6

Итоги

- Узнали, что такое ConfigMap, Secret и чем они отличаются
- Разобрались со способами подключения и применения конфигураций
- Узнали, что такое ServiceAccount, Роли и RBAC
- Разобрались с подключением Ролей и пользователей
- Поняли, как можно ограничить права доступа пользователей и приложений к API кластера
- Рассмотрели примеры манифестов объектов K8s
- Подключились к кластеру и посмотрели в работе объекты, изученные на занятии



Настройка приложений и управление доступом

Кирилл Касаткин
DevOps-инженер, Renue

